

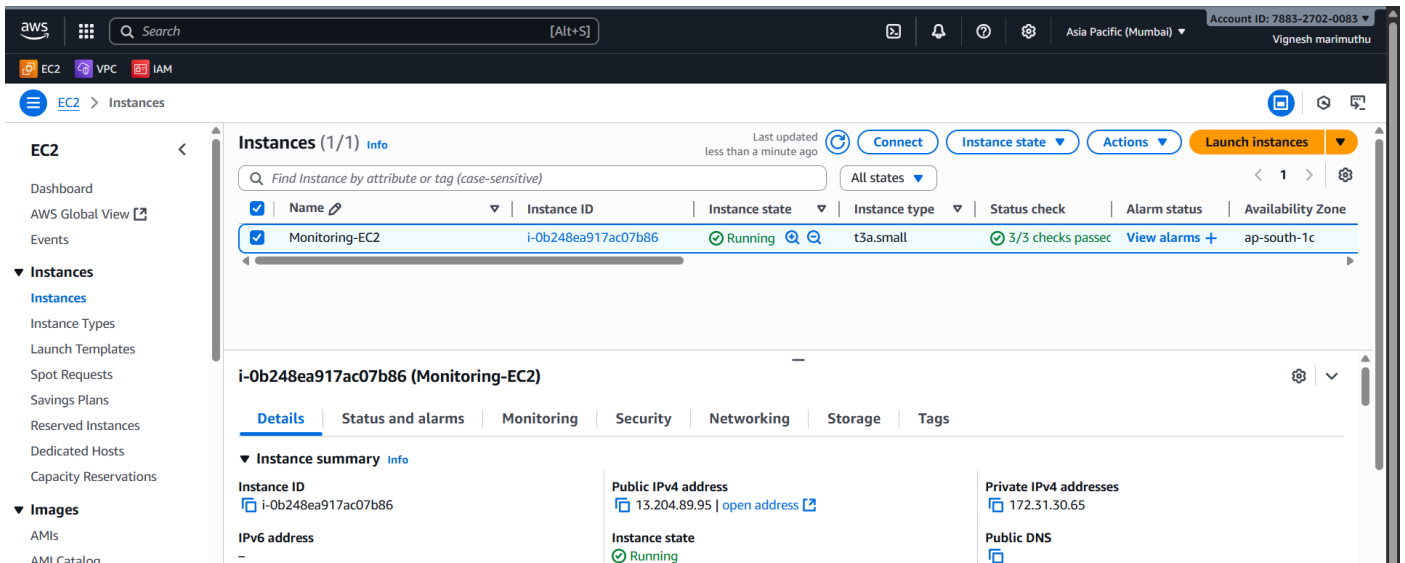
Monitoring Task

Task Description:

Install Prometheus and Grafana on a Linux EC2 machine, connect Prometheus to Grafana, and create a dashboard to view metrics.

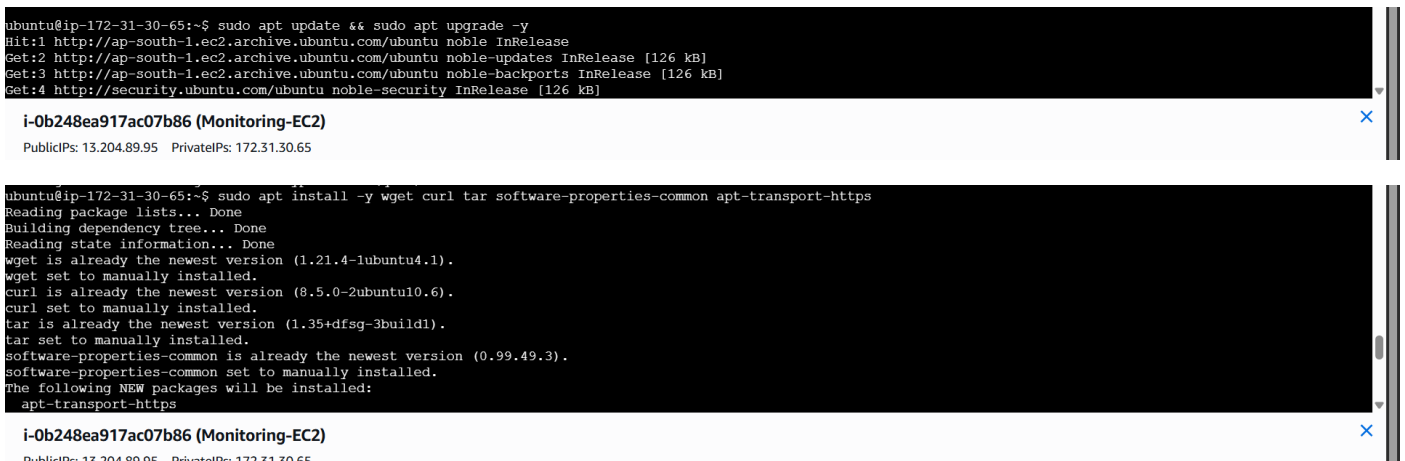
Solution:

✓ 1. Launch an EC2 Instance & Connect to EC2



- Creates the environment (EC2 server) where all monitoring tools will run.

✓ 2. Update and Install Basic Tools



- Update the system to the latest security patches and install tools (wget, curl, tar) which are needed to download and extract Prometheus/Grafana packages.

✓ 3. Install Prometheus

```
ubuntu@ip-172-31-30-65:~$ sudo useradd --no-create-home --shell /bin/false prometheus
ubuntu@ip-172-31-30-65:~$ sudo mkdir -p /etc/prometheus /var/lib/prometheus
ubuntu@ip-172-31-30-65:~$ cd /tmp
wget https://github.com/prometheus/prometheus/releases/download/v2.54.1/prometheus-2.54.1.linux-amd64.tar.gz
tar -xvzf prometheus-2.54.1.linux-amd64.tar.gz
sudo cp prometheus-2.54.1.linux-amd64/prometheus /usr/local/bin/
sudo cp prometheus-2.54.1.linux-amd64/promtool /usr/local/bin/
sudo cp -r prometheus-2.54.1.linux-amd64/consoles /etc/prometheus/
sudo cp -r prometheus-2.54.1.linux-amd64/console_libraries /etc/prometheus/
--2025-10-03 06:59:13-- https://github.com/prometheus/prometheus/releases/download/v2.54.1/prometheus-2.54.1.linux-amd64.tar.gz
Resolving github.com (github.com)... 20.207.73.82
Connecting to github.com (github.com)|20.207.73.82|:443... connected.
```

i-Ob248ea917ac07b86 (Monitoring-EC2)

PublicIPs: 13.204.89.95 PrivateIPs: 172.31.30.65

- A dedicated user (prometheus) and folders are created so Prometheus doesn't run as root (safer).
- This improves security and keeps files organized.
- Installs Prometheus so we can start collecting metrics.

✓ 4. Configure Prometheus

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['localhost:9100']
```

- Defines what data Prometheus should collect.

✓ 5. Start Prometheus Service

```
[Unit]
Description=Prometheus
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus/ \
  --web.console.templates=/etc/prometheus/consoles \
  --web.console.libraries=/etc/prometheus/console_libraries \
  --web.listen-address=0.0.0.0:9090

Restart=on-failure

[Install]
WantedBy=multi-user.target
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-_ To Bracket M-^ Previous
^X Exit ^R Read File ^N Replace ^V Paste ^J Justify ^_ Go To Line M-E Redo M-G Copy M-^ Where Was M-W Next

i-Ob248ea917ac07b86 (Monitoring-EC2)

PublicIPs: 13.204.89.95 PrivateIPs: 172.31.30.65

```

ubuntu@ip-172-31-30-65:/tmp$ sudo systemctl daemon-reload
sudo systemctl enable prometheus
sudo systemctl start prometheus
sudo systemctl status prometheus
Created symlink /etc/systemd/system/multi-user.target.wants/prometheus.service -> /etc/systemd/system/prometheus.service.
● prometheus.service - Prometheus
   Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-10-03 07:01:03 UTC; 41ms ago
   Main PID: 9132 (prometheus)
     Tasks: 4 (limit: 2057)
    Memory: 8.6M (peak: 8.6M)
       CPU: 36ms
    CGroup: /system.slice/prometheus.service
            └─9132 /usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.path=/var/lib/prometheus

Oct 03 07:01:03 ip-172-31-30-65 systemd[1]: Started prometheus.service - Prometheus.
ubuntu@ip-172-31-30-65:/tmp$

```

i-Ob248ea917ac07b86 (Monitoring-EC2)

PublicIPs: 13.204.89.95 PrivateIPs: 172.31.30.65

- We use systemd to manage Prometheus, so it runs automatically after reboot.
- Keeps Prometheus running continuously as a background service.

✓ 6. Install Node Exporter

```

[Unit]
Description=Node Exporter
After=network.target

[Service]
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target

```

```

ubuntu@ip-172-31-30-65:/tmp$ sudo nano /etc/systemd/system/node_exporter.service
ubuntu@ip-172-31-30-65:/tmp$ sudo systemctl daemon-reload
sudo systemctl start node_exporter
sudo systemctl status node_exporter
Created symlink /etc/systemd/system/multi-user.target.wants/node_exporter.service -> /etc/systemd/system/node_exporter.service.
ubuntu@ip-172-31-30-65:/tmp$

```

i-Ob248ea917ac07b86 (Monitoring-EC2)

PublicIPs: 13.204.89.95 PrivateIPs: 172.31.30.65

```

ubuntu@ip-172-31-30-65:/tmp$ curl -s http://13.204.89.95:9100/metrics | head -n 10
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
go_gc_duration_seconds_count 0
# HELP go_goroutines Number of goroutines that currently exist.
ubuntu@ip-172-31-30-65:/tmp$

```

i-Ob248ea917ac07b86 (Monitoring-EC2)

PublicIPs: 13.204.89.95 PrivateIPs: 172.31.30.65

- Node Exporter exposes system metrics (CPU, memory, disk, network) at port 9100.
- It provides the actual EC2 metrics that Prometheus scrapes.

✓ 7. Install Grafana

```

ubuntu@ip-172-31-30-65:/tmp$ wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee /etc/apt/sources.list.d/grafana.list
sudo apt update
sudo apt install -y grafana
sudo systemctl enable grafana-server
sudo systemctl start grafana-server
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instead (see apt-key(8)).
OK
deb https://packages.grafana.com/oss/deb stable main
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:4 https://packages.grafana.com/oss/deb stable InRelease [7660 B]
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:6 https://packages.grafana.com/oss/deb stable/main amd64 Packages [435 kB]

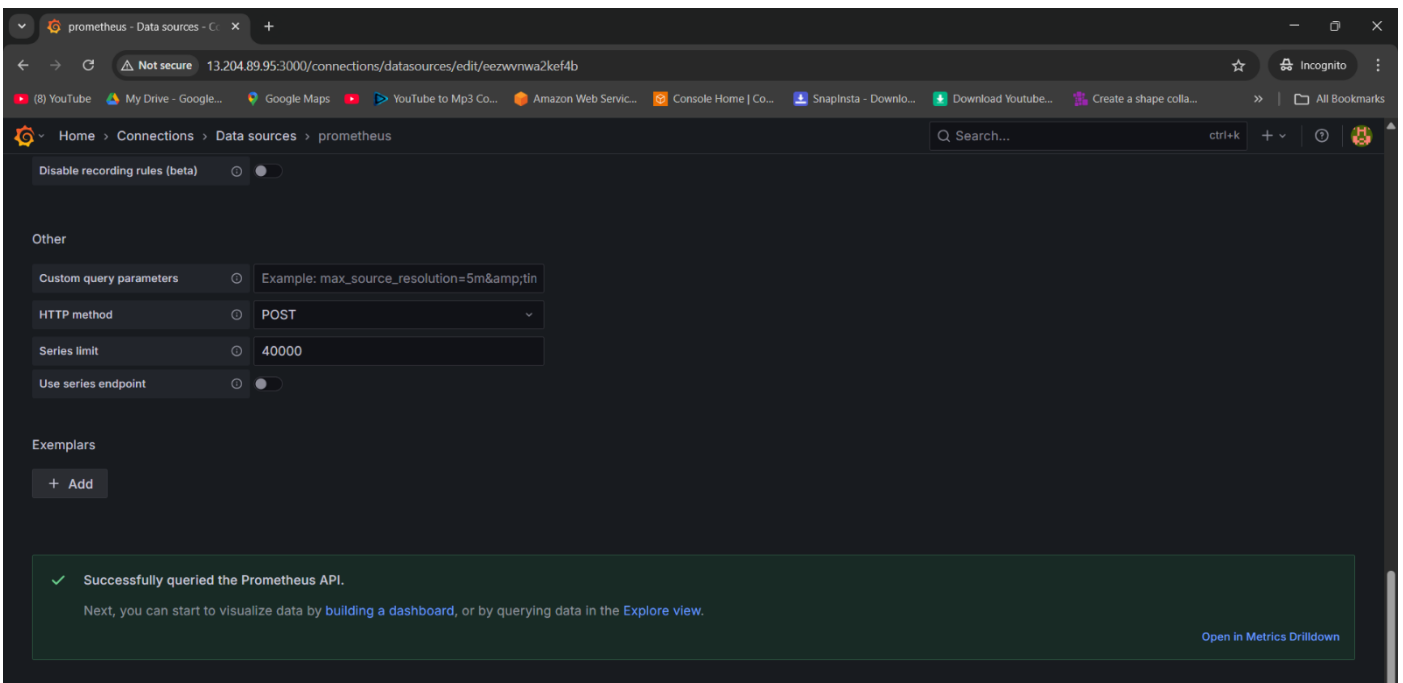
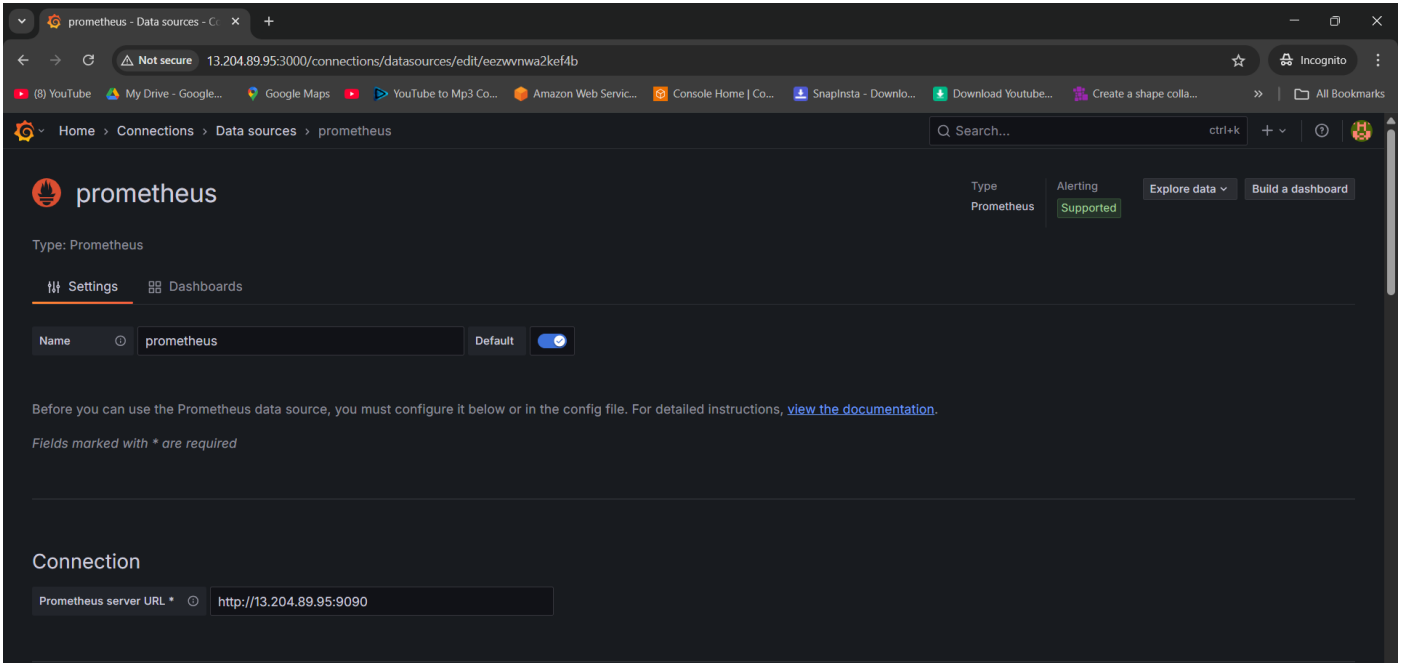
```

i-Ob248ea917ac07b86 (Monitoring-EC2)

PublicIPs: 13.204.89.95 PrivateIPs: 172.31.30.65

- Grafana is installed from the official repository and started as a service.
- Grafana lets us visualize Prometheus metrics with dashboards.

✅ 8. Connect Prometheus to Grafana



- Grafana needs to know where Prometheus is running.
- This connects Grafana to Prometheus to fetch metrics.

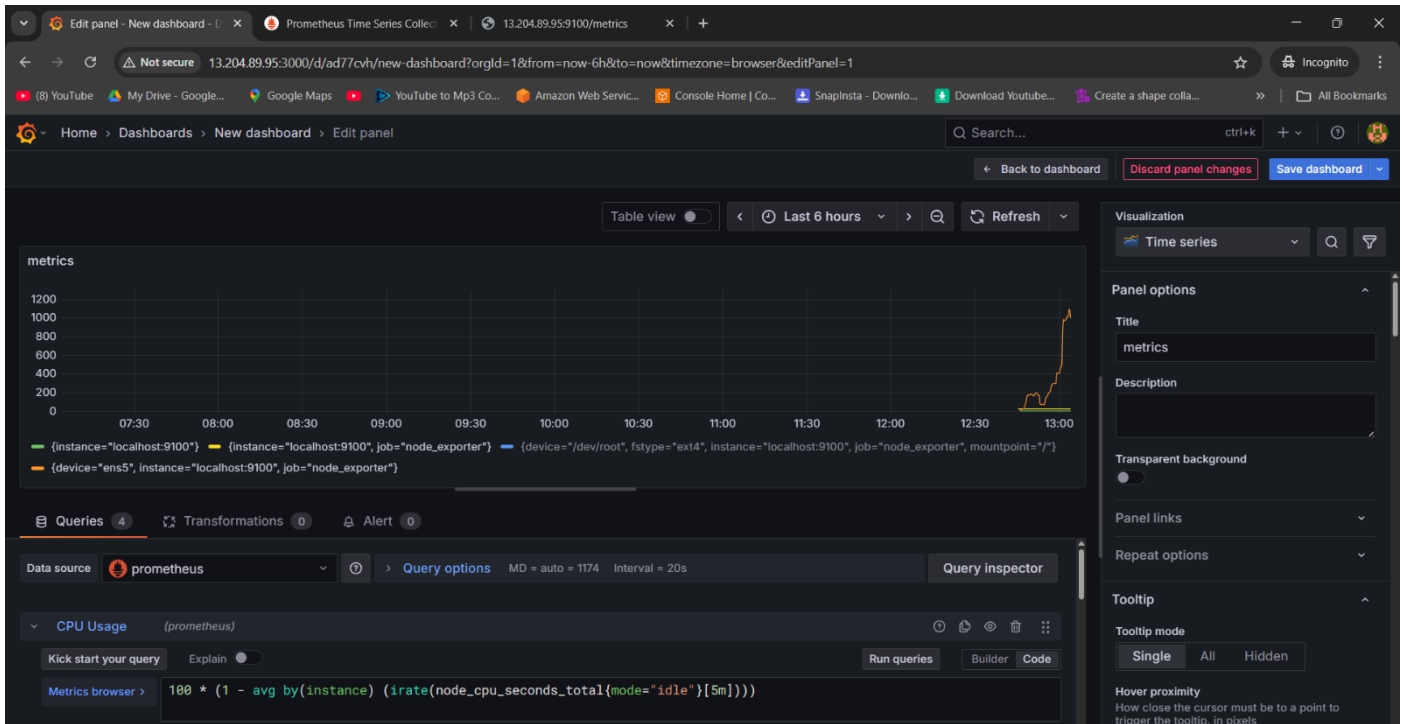
✓ 9. Create Grafana Dashboard

This screenshot shows the Grafana dashboard editor interface. The top navigation bar includes 'Home', 'Dashboards', 'New dashboard', and 'Edit panel'. The main area displays two queries for 'CPU Usage' and 'Memory Usage' using Prometheus as the data source. The 'CPU Usage' query is `100 * (1 - avg by(instance) (irate(node_cpu_seconds_total{mode="idle"}[5m])))` and the 'Memory Usage' query is `100 * (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes))`. The right sidebar contains settings for 'Visualization' (Time series), 'Repeat options', 'Tooltip' (Single mode), and 'Legend' (Visibility on).

This screenshot shows the Grafana dashboard editor interface with two more queries. The 'Disk Usage' query is `100 * (((node_filesystem_size_bytes{mountpoint="/" } - node_filesystem_free_bytes{mountpoint="/"}) / node_filesystem_size_bytes{mountpoint="/"}))` and the 'Network Traffic' query is `rate(node_network_receive_bytes_total{device!="lo"}[5m])`. The right sidebar shows settings for 'Visualization' (Time series), 'Repeat options', 'Tooltip' (Single mode), and 'Legend' (Visibility on, Mode: List).

- These are PromQL queries that calculate resource usage.
- Turns raw metrics into visual dashboards.

9. Verify



The screenshot shows the Prometheus 'Targets' page. The table lists the scrape targets, their states, and labels. The first target is 'node_exporter (1/1 up)' with a state of 'UP'. The second target is 'prometheus (1/1 up)' with a state of 'UP'. The table includes columns for Endpoint, State, Labels, Last Scrape, Scrape Duration, and Error.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9100/metrics	UP	<code>instance="localhost:9100"</code> <code>job="node_exporter"</code> Discovered labels: <code>_address_="/localhost:9100"</code> <code>_metrics_path_="/metrics"</code> <code>_scheme_="http"</code> <code>_scrape_interval_="15s"</code> <code>_scrape_timeout_="10s"</code> <code>job="node_exporter"</code>	1m 39s ago	17.229ms	
http://localhost:9090/metrics	UP	<code>instance="localhost:9090"</code> <code>job="prometheus"</code> Discovered labels: <code>_address_="/localhost:9090"</code>	1m 40s ago	6.521ms	

```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 2.789e-05
go_gc_duration_seconds{quantile="0.25"} 4.5561e-05
go_gc_duration_seconds{quantile="0.5"} 4.889e-05
go_gc_duration_seconds{quantile="0.75"} 5.6152e-05
go_gc_duration_seconds{quantile="1"} 8.5431e-05
go_gc_duration_seconds_sum 0.002444042
go_gc_duration_seconds_count 47
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 8
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.22.5"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.66088e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 8.5097168e+07
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.47786e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 1.071574e+06
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 1.13688e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
```

- Verification ensures each service is running correctly
-